

Midterm 1 Practice — Random Walks on Graphs

M 375T Mathematical Foundations of Machine Learning — Fall 2026

released Tuesday, September 29 · solutions posted Thursday, October 1

What the midterm looks like. Tuesday, October 6, in class. 75 minutes. Closed book, no devices, no formula sheet. Four questions of 10 points each (40 points). Every question starts with a short NumPy program (10–20 lines) that you must read without running it; the parts ask you to predict what it prints and to justify the prediction by hand. The four questions are: (1) a walk from code, (2) the best Gaussian by maximum likelihood, (3) the Laplacian and the lazy walk on a small graph, (4) the two-bin test on a count of samples (as in Quiz 3 and Quiz 4).

Three of the four midterm questions are on this sheet, word for word: A1, D4 and F4. The fourth (the best Gaussian) is F3 with different numbers. If you can do A1, D4, F3 and F4 without looking anything up, and can explain every line of your answer, you are ready. The other exercises build the pieces those four use.

What the midterm covers. Lectures 7–10 and Practice 5 (graphs, $W \rightarrow d \rightarrow P$, laws and $P^T \mu$, the Markov property, the invariant measure, the Laplacian and its eigenvectors, the lazy walk and its convergence); maximum likelihood for a Gaussian (Lecture 3 §5, Lecture 4 §1–§2, Practice 3 A1–A2); the two-bin test (Lecture 5, Practice 4 A1–A4, B1–B3, Quizzes 3–4). Lecture 11 is not on the midterm. No proofs are asked as stand-alone questions; the proof exercises below (A4, B2, C1, D1, D3, E3) are the reasons behind the numbers you will compute.

Every NumPy command on the midterm appears in Practice 2–5 or on this sheet:

`np.array`, `W.sum(axis=1)`, `W / d[:, None]`, `P.T @ mu`, `P @ P`, `np.ones`, `np.diag`, `v @ L @ v`, `np.eye`, `np.linalg.eigvalsh` (eigenvalues of a symmetric matrix, increasing order), `np.round`, `np.linspace`, `np.argmax`, `x.mean()`, `x.std()`, `np.abs`, `rng.uniform`, `(cond).sum(axis=1)`, `np.mean(cond)`.

Conventions. $P_{ij} = W_{ij}/d_i$ (rows sum to one); laws are column vectors, $\mu_{t+1} = P^T \mu_t$; $Q = \frac{1}{2}(I + P)$; $L = D - W$. Do everything on paper first, then check in NumPy.

Part A — Graphs, walks, laws

(midterm Q1)

A1. A walk from code. [= midterm Q1]

[10 pts]

```
import numpy as np
W = np.array([[0, 1, 2, 0],
              [1, 0, 1, 0],
              [2, 1, 0, 1],
              [0, 0, 1, 0]], dtype=float)
d = W.sum(axis=1)
P = W / d[:, None]

mu = np.array([0., 0., 0., 1.])
for t in range(2):
    mu = P.T @ mu
print(mu)                                # line 1
print((P @ P)[3, 3])                    # line 2
```

```
u = np.ones(4) / 4
print(P.T @ u)                                # line 3
```

- (a) Write d and P .
- (b) Predict line 1. Say in one sentence what the vector μ is at the moment it is printed.
- (c) Predict line 2. Which routes of the walker does this number add up?
- (d) Write, from memory, the equation an invariant measure π satisfies, and find π for this graph. Then predict entry 3 of line 3 (the last entry) and say what it shows about the uniform vector u .

A2. A weighted path. Vertices 0, 1, 2; edges 0–1 with weight 1 and 1–2 with weight 3. [8 pts]

- (a) Write W , d , P , and $Q = \frac{1}{2}(I + P)$.
- (b) Predict the output of the code. Do not run it.

```
mu = np.array([1., 0., 0.])
for t in range(3):
    mu = P.T @ mu
    print(mu)
print((P @ P)[0, 0])
print(Q[1])
```

- (c) Explain in one sentence what the number $(P @ P)[0, 0]$ is, as a probability. List the routes that contribute to it.
- (d) Explain why $P @ \mu$ would be wrong for moving a law. Give the check that catches the mistake.

A3. The Markov property. Same graph as A2. The walker starts at vertex 2. [6 pts]

- (a) State the Markov property as an equation between two conditional probabilities.
- (b) Compute $\mathbb{P}(S_2 = 2 \mid S_1 = 1)$ and $\mathbb{P}(S_2 = 2 \mid S_1 = 1, S_0 = 2)$. Why are they equal?
- (c) Compute $\mathbb{P}(S_2 = 2)$ by the law of total probability over S_1 .
- (d) Markov is not independence: compute $\mathbb{P}(S_2 = 2 \mid S_1 = 2)$ and compare with $\mathbb{P}(S_2 = 2)$.

A4. Derive the update rule. Let (S_t) be a Markov chain with transition matrix P and law $\mu_t(i) = \mathbb{P}(S_t = i)$. Prove $\mu_{t+1} = P^T \mu_t$ in three lines. Say where the law of total probability is used and where the Markov property is used. Explain why the sum runs over the source vertex, and why that puts the transpose on P . [4 pts]

A5. The star. Vertices 0, 1, 2, 3. Vertex 0 is joined to each of 1, 2, 3 with weight 1; there are no other edges. [8 pts]

- (a) Write d , P , π .
- (b) Start at vertex 1. Write the ordinary laws $\mu_0, \mu_1, \mu_2, \mu_3$. Which two laws alternate? Why does the ordinary law never converge on this graph?
- (c) Write the lazy laws for $t = 0, 1, 2$. What do they converge to?
- (d) Give an eigenvector of P with eigenvalue -1 . Check it by multiplying. What is its eigenvalue for Q ?

Part B — The invariant measure

(midterm Q1(d))

B1. Fixed point of the update. Graph of A2.

[8 pts]

- (a) Write down, from memory, the equation for an invariant measure and the two conditions that make it a probability vector.
- (b) Find π . Verify the equation at every vertex by explicit arithmetic.
- (c) Prove that $Q^T \pi = \pi$ follows from $P^T \pi = \pi$.
- (d) Predict the output of `u = np.ones(3) / 3; print(P.T @ u)`. Is u invariant? In one sentence: why must vertex 1 carry more mass than vertex 0 in equilibrium?

B2. Detailed balance. For any undirected graph with positive degrees, let $s = \sum_i d_i$ and $\pi_i = d_i/s$.

[4 pts]

- (a) Show $\pi_i P_{ij} = \pi_j P_{ji}$ for all i, j . Where is $W = W^T$ used?
- (b) Sum the identity of (a) over i to obtain $P^T \pi = \pi$.

B3. Read it off the walker. A long simulation of the lazy walk on the star of A5 spends the fraction 0.50 of its time at vertex 0 and about 0.17 at each leaf. Explain this with a formula. Then say in one sentence what π describes about a long walk, and why the endpoint law μ_t of the *ordinary* walk on the star does not give the same picture.

[3 pts]

Part C — Eigenvalues of P

(background for Q3)

C1. The bound. Prove that every eigenvalue λ of a transition matrix P satisfies $|\lambda| \leq 1$. Then show that $\lambda = 1$ always occurs. Then explain why $\lambda = -1$ occurs on the four-vertex path $0-1-2-3$ (give the eigenvector) and why it cannot occur for the lazy matrix Q .

[6 pts]

C2. Which are eigenvectors? For the star graph of A5, decide for each of $v = (1, 1, 1, 1)^T$, $w = (0, 1, -1, 0)^T$, $u = (1, -1, -1, -1)^T$, $z = (1, 0, 0, 0)^T$ whether it is an eigenvector of P ; for each one that is, give the eigenvalue, and give the corresponding eigenvalue of Q and of the generalized Laplacian problem $Lv = \lambda Dv$. Justify each answer by multiplying.

[6 pts]

Part D — The Laplacian, energy, and the lazy walk

(midterm Q3)

D1. The Dirichlet form. Prove $v^T L v = \frac{1}{2} \sum_{i,j} W_{ij} (v_i - v_j)^2$ for $L = D - W$. Deduce: (i) every eigenvalue of L is ≥ 0 ; (ii) $Lv = 0$ if and only if v is constant on every connected component; (iii) $\dim \ker L$ equals the number of components.

[6 pts]

D2. Two triangles and a bridge. Vertices $0, \dots, 5$; edges of weight 1 inside each triangle $\{0, 1, 2\}$ and $\{3, 4, 5\}$; one bridge $2-3$ of weight 1. W is its weight matrix.

[8 pts]

```
d = W.sum(axis=1)
L = np.diag(d) - W
v = np.array([1., 1., 1., -1., -1., -1.])
print(v @ L @ v)           # line 1
print(L @ v)               # line 2
print(np.linalg.eigvalsh(L)[0]) # line 3
```

- (a) Write d , π , and row 2 of P and of Q .
- (b) Predict line 1 with the Dirichlet form. Which edges contribute?
- (c) Predict line 2. Is v an eigenvector of L ? Which entries break it, and why are the bridge vertices the problem?
- (d) Predict line 3 and say why. Check that $(1, -1, 0, 0, 0)^T$ is an eigenvector of L and give its eigenvalue.
- (e) Compute $\text{cut}(A, B)$, $\text{vol}(A)$, $\text{vol}(B)$ and $\text{Ncut}(A, B) = \text{cut}(A, B) \left(\frac{1}{\text{vol}(A)} + \frac{1}{\text{vol}(B)} \right)$ for $A = \{0, 1, 2\}$, $B = \{3, 4, 5\}$. In two sentences: why does the sign of v_2 (the second eigenvector of $Lv = \lambda Dv$) recover the two triangles?

D3. The identity $D^{-1}L = I - P$. Prove it. Then show: $Lv = \lambda Dv \iff Pv = (1 - \lambda)v \iff Qv = (1 - \frac{\lambda}{2})v$. For a graph in which every vertex has the same degree δ , deduce $Q = I - \frac{1}{2\delta}L$ and say why P and Q are then symmetric. [4 pts]

D4. The Laplacian and the lazy walk. [= midterm Q3] [10 pts]

```
W = np.array([[0, 3, 0, 1],
              [3, 0, 1, 0],
              [0, 1, 0, 3],
              [1, 0, 3, 0]], dtype=float)
d = W.sum(axis=1)
L = np.diag(d) - W
v = np.array([1., 1., -1., -1.])
print(L @ v)           # line 1
print(v @ L @ v)       # line 2
print(np.linalg.eigvalsh(L)) # line 3

P = W / d[:, None]
Q = (np.eye(4) + P) / 2
print(np.linalg.eigvalsh(Q)) # line 4
mu = np.array([1., 0., 0., 0.])
for t in range(12):
    mu = Q.T @ mu
print(np.round(mu, 3))      # line 5
```

- (a) Draw the graph with its edge weights, and write d and L .
- (b) Predict lines 1 and 2. Check line 2 with the Dirichlet form $v^T Lv = \frac{1}{2} \sum_{i,j} W_{ij} (v_i - v_j)^2$, naming the edges that contribute.
- (c) The vectors $(1, 1, 1, 1)^T$, v , $(1, -1, -1, 1)^T$, $(1, -1, 1, -1)^T$ are all eigenvectors of L . Compute their eigenvalues and predict line 3.
- (d) Predict line 4. Use the identity $D^{-1}L = I - P$ to write $Q = I - cL$ with the right constant c . Why is `eigvalsh` allowed on Q here?
- (e) Write π and $\mu_0 = e_0$ as a combination of the four eigenvectors of (c), write μ_t as a formula in t , and predict line 5. You may use $(\frac{3}{4})^{12} \approx 0.0317$.

Part E — Convergence and the power method

(midterm Q3(d)–(e))

E1. The lazy walk on the 4-cycle, from code. The graph is the unweighted 4-cycle 0–1–2–3–0, all weights 1. [8 pts]

```
W = np.array([[0, 1, 0, 1],
              [1, 0, 1, 0],
              [0, 1, 0, 1],
              [1, 0, 1, 0]], dtype=float)
d = W.sum(axis=1)
L = np.diag(d) - W
P = W / d[:, None]
Q = (np.eye(4) + P) / 2
print(np.linalg.eigvalsh(L))      # line 1
print(np.linalg.eigvalsh(Q))      # line 2
mu = np.array([1., 0., 0., 0.])
for t in range(3):
    mu = Q.T @ mu
print(np.round(mu, 3))           # line 3
```

- (a) Check that $(1, 1, 1, 1)$, $(1, 0, -1, 0)$, $(0, 1, 0, -1)$, $(1, -1, 1, -1)$ are eigenvectors of L and compute their eigenvalues. Predict line 1.
- (b) Why is Q symmetric for this graph, so that `eigvalsh` is allowed? Write $Q = I - cL$ with the right constant c and predict line 2.
- (c) Write e_0 as a combination of the four eigenvectors, write μ_t as a formula in t , and predict line 3.
- (d) Which eigenvector's term is gone after a single step, and what does that say about the ordinary walk on this graph?

E2. The power method by hand. Let $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ and $x_0 = (1, 0)^\top$. [6 pts]

- (a) Find the eigenvalues and eigenvectors of A .
- (b) Compute $x_1 = Ax_0$ and $x_2 = Ax_1$ (no need to normalize). Compute the Rayleigh quotient $R(x_t) = x_t^\top Ax_t / x_t^\top x_t$ for $t = 0, 1, 2$. What does it converge to?
- (c) By what factor per step does the angle between x_t and the top eigenvector shrink? Explain using the expansion of x_0 in the eigenbasis.
- (d) The walk's law $\mu_t = (Q^\top)^t \mu_0$ is a power method. For which matrix, and which eigenvector does it converge to? Why is no normalization step needed?

E3. Why the lazy walk converges. Using the expansion $\mu_t = \pi + \sum_{k \geq 2} a_k \eta_k^t w_k$: state which two facts about the η_k are needed for $\mu_t \rightarrow \pi$, and say which property of the graph gives each fact (connectedness; laziness). Then explain in one sentence why $a_1 w_1 = \pi$ for every starting law μ_0 . [4 pts]

Part F — Cleaning, maximum likelihood, the two-bin test (midterm Q2 and Q4)

F1. Standardize. A feature has sample mean 10 and sample standard deviation 4. What does $x = 16$ map to? Which x maps to $z = -0.5$? After standardizing, what is the only Gaussian candidate left, and why? [3 pts]

F2. The two-bin test by hand. In $n = 400$ samples, $k = 230$ land in the inner bin where the model predicts $\frac{1}{2}$. Compute $\varepsilon = k/n - \frac{1}{2}$, $D \approx 2\varepsilon^2$, and $2nD$. Does the test at the 5% level (reject when $2nD > 3.84$) reject the model? Rewrite the rule as a rule on k : for which counts k out of 400 does the test reject? ($\sqrt{3.84} = 1.96$.) [4 pts]

F3. Maximum likelihood for a Gaussian, from code. [midterm Q2 has this form with other numbers] [6 pts]

```
x = np.array([1., 3., 5., 7., 9.])
sig = 1.0
ms = np.linspace(0, 10, 201)
H = [np.mean(0.5*np.log(2*np.pi*sig**2) + (x - m)**2 / (2*sig**2))
      for m in ms]
print(ms[np.argmin(H)])          # line 1
print(min(H))                   # line 2
z = (x - x.mean()) / x.std()
print(z)                        # line 3
print(np.mean(-np.log(np.exp(-z**2/2) / np.sqrt(2*np.pi)))) # line 4
```

- Write $H(m)$ as a formula and name it. Predict line 1 by computing $dH/dm = 0$. Would the answer change for $\text{sig} = 3.0$?
- Predict line 2 as an exact expression: substitute $m = \bar{x}$ and use $\frac{1}{n} \sum_i (x_i - \bar{x})^2 = \hat{\sigma}^2$ (compute $\hat{\sigma}^2$ for these five numbers).
- Predict lines 3 and 4. Why is line 4 equal to $\frac{1}{2} \log(2\pi) + \frac{1}{2} \approx 1.419$ for every dataset? What does that say about the cross-entropy score as a judge of shape?
- Now let both m and σ vary. Show that the minimizer of $H(m, \sigma) = \frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \cdot \frac{1}{n} \sum_i (x_i - m)^2$ is $m = \bar{x}$, $\sigma^2 = \hat{\sigma}^2$, and that the minimum value is $\frac{1}{2} \log(2\pi e \hat{\sigma}^2)$.

F4. Is the model right? [= midterm Q4] Each row of $\mathbf{X}$ is one data set of $n = 400$ independent samples from the uniform distribution on $[-2, 2]$. [10 pts]

```
rng = np.random.default_rng(0)
X = rng.uniform(-2, 2, size=(50_000, 400))
k = (np.abs(X) < 1).sum(axis=1)
print(k.shape)                  # line 1
print(k.mean() / 400)          # line 2
print(k.std())                  # line 3
eps = k / 400 - 0.5
D = 2 * eps**2
print(np.mean(2 * 400 * D > 3.84)) # line 4
```

- (a) Predict line 1. Say in one sentence what one entry of $\mathbf{k}$ counts.
- (b) Predict line 2 (the value it is close to). Justify it with the probability that one sample lands in $(-1, 1)$.
- (c) Predict line 3 (the value it is close to). Name the distribution of one entry of $\mathbf{k}$ and use its variance.
- (d) Predict line 4 (the value it is close to). Which theorem says so, and what is the role of the number 3.84?
- (e) A different experiment produces one data set of 400 samples from an unknown distribution, and 170 of them land in $(-1, 1)$. Compute ε , D and $2nD$ for this data set. Is it consistent with the uniform distribution on $[-2, 2]$?

Total on this sheet: 131 points, about 3 times the midterm. A1, D4, F4 are midterm questions 1, 3, 4 word for word; F3 is midterm question 2 with other numbers. Solutions will be posted after Thursday's class. Bring questions to office hours (Tue/Thu 5–6 PM, PMA 9.156).